Объектно-ориентированное программирование

Лекция 2. Переход к Си++ и знакомство с ООП

Переход к Си++. Перегрузка функций

```
/* Код на Си */

int sumi(int a, int b);

float sumf(float a, float b);

int main() {
  … = sumi(1, 3);
  … = sumf(1.0f, 3.0f);
}
```

```
/* Код на Си++ */

int sum(int a, int b);

float sum(float a, float b);

int main() {
  … = sum(1, 3);
  … = sum(1.0f, 3.0f);
}
```

Переход к Си++. Ключевое слово nullptr

```
/* Код на Си++ */

void print(int value); // 1
void print(void* ptr); // 2

int main() {
  int v = 42;
  print(v);
  print(&v);
  print(NULL);
  print(nullptr);
}
```

⚠ В C++ вместо `NULL` всегда должен использоваться `nullptr`

Переход к Си++. Пространства имён

```
#include <stdio.h>

namespace g2d {
  struct Point {
    float x;
    float y;
  };
}

namespace g3d {
  struct Point {
    float x;
    float y;
    float z;
  };
}
```

```
// Вариант использования
// Допускается в .h и .cc
g2d::Point p2;
g3d::Point p3;

// Вариант использования
// Допускается только в .cc
using namespace g2d;
Point p;
```

⚠ `using namespace` не должен использоваться в заголовочных файлах

Переход к Си++. Пространства имён: определение функции

```
/* sum.h */

namespace my_library {

  int sum(int a, int b);

}
```

```
/* sum.cc */

#include "sum.h"

using namespace my_library;

int my_library::sum(int a, int b) {
  …
}
```

Переход к Си++. Использование стандартной библиотеки Си

cstdio~~.h~~

```
/* Код на Си */

#include <stdio.h>

int main() {
  puts("Hello, world!");
}
```

```
/* Код на Си++ */

#include <cstdio>

using namespace std;

int main() {
  puts("Hello, world!");
}
```

Переход к Си++. Ссылки

```
/* Код на Си */

void inc(int* value_ptr) {
  ++(*value_ptr);
}

int main() {
  int v = 1;
  inc(&v);
}
```

```
/* Код на Си++ */

void inc(int& value) {
  ++value;
}

int main() {
  int v = 1;
  inc(v);
}
```

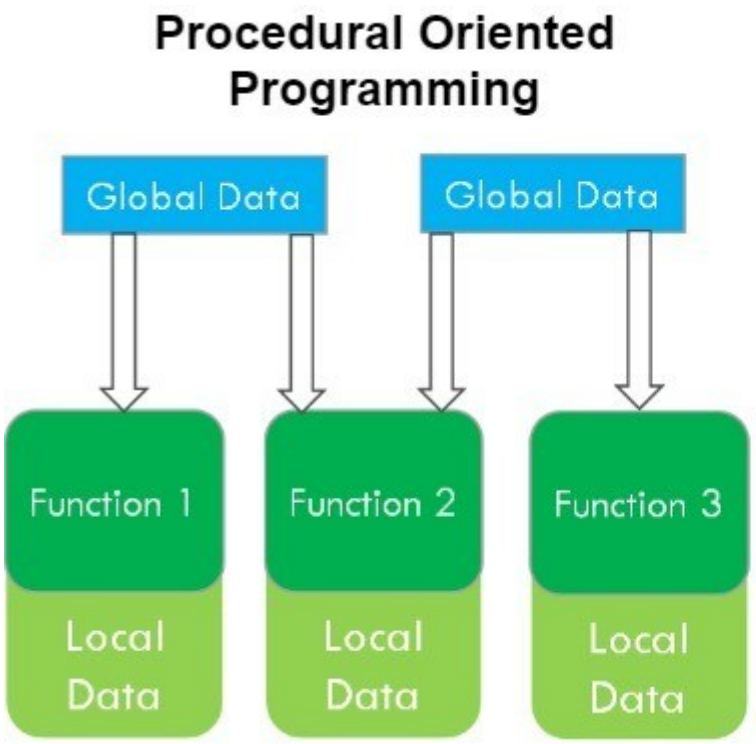
Указатель

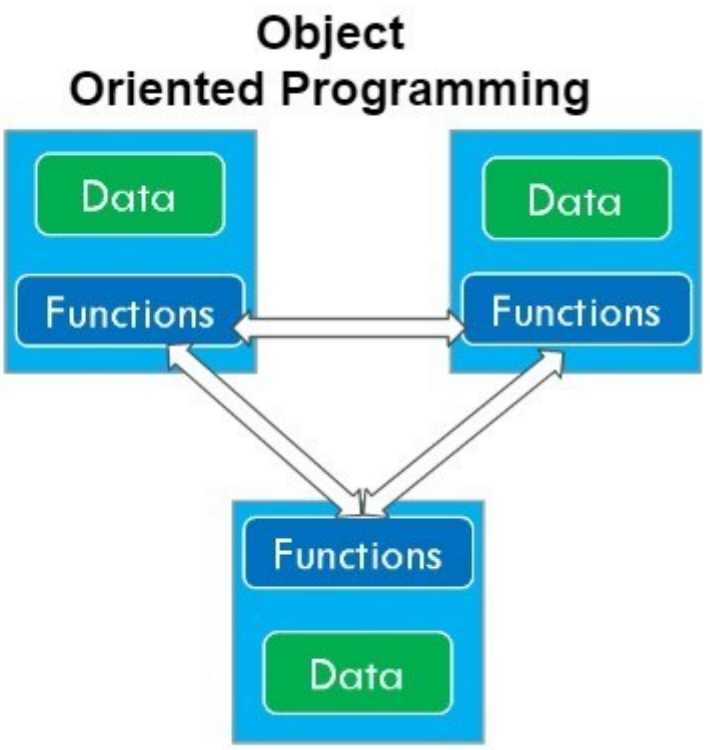
1. Может быть `nullptr`
2. Неконстантный указатель может быть перекинут на другие данные

Ссылка

1. Не может быть `nullptr`
2. Не может быть перекинута на другой объект

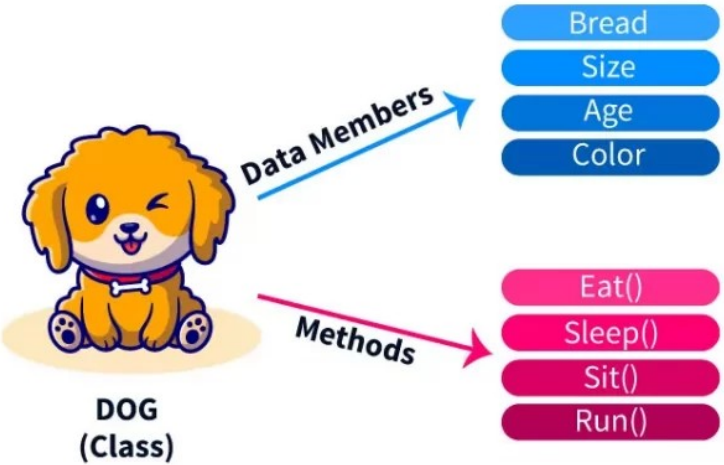
Объектно-ориентированная парадигма программирования





[Object oriented programming concepts]

Объектно-ориентированная парадигма программирования



Difference between object and class

Класс (тип) – это способ описания сущности, определяющий состояние и поведение, зависящее от этого состояния.

С точки зрения программирования класс можно рассматривать как набор данных (полей, атрибутов) и функций для работы с ними (методов).

Объект (экземпляр) – это отдельный представитель класса, имеющий конкретное состояние и поведение, полностью определяемое классом.

Объектно-ориентированная парадигма программирования

Пример 1

Класс: массив

Состояние (**поля**):
– массив
– количество элементов

Поведение (**методы**):
– узнать количество элементов
– получить элемент по индексу
– вставить элемент по индексу
– удалить элемент по индексу

Пример 2

Класс: квадратная матрица

Состояние (**поля**):
– элементы матрицы
– размер матрицы

Поведение (**методы**):
– узнать размер матрицы
– получить элемент по индексам
– установить элемент по индексам
– вычислить детерминант
– найти обратную матрицу
– умножить матрицу на число
– умножить матрицу на матрицу

Объектно-ориентированная парадигма программирования

Пример 3

Класс: счёт в банке

Состояние (**поля**):
– владелец счета
– баланс счёта

Поведение (**методы**):
– списать сумму
– зачислить сумму
– рассчитать проценты на остаток

Пример 4

Класс: игровой персонаж

Состояние (**поля**):
– имя
– количество жизни
– количество маны
– броня

Поведение (**методы**):
– применить “buff”
– нанести урон
– получить урон

ООП на Си++. Методы

```
#include <cmath>

struct Complex {
  float re;
  float im;

  float abs() {
    return std::sqrt(
      std::pow(this->re, 2) +
      std::pow(this->im, 2)
    );
  }

  Complex sum(Complex rhs) {
    Complex r = {
      .re = this->re + rhs.re,
      .im = this->re + rhs.im
    };
    return r;
  }
};
```

```
…

int main() {
  Complex a = {.re = 3.0f, .im = 4.0f};
  Complex b = {.re = 0.0f, .im = 1.0f};

  printf("%.2f\n", a.abs());

  Complex c = b.sum(a);
  printf("%.2f %.2f\n", c.re, c.im);
}
```

`this` – указатель на объект (экземпляр), для которого этот метод был вызван.

При реализации метода `this` в большинстве случаев можно опускать: вместо `this->re` писать просто `re`.

ООП на Си++. Конструкторы

```
struct Complex {
  float re;
  float im;

  // Конструктор по умолчанию (1)
  Complex() {
    re = 0;
    im = 0;
  }

  // Конструктор с параметрами (2)
  Complex(float re, float im) {
    this->re = re;
    this->im = im;
  }
};

int main() {
  Complex c1;              // (1)
  Complex c2(3.0f, 4.0f); // (2)
}
```

Если в классе не объявлено ни одного конструктора, то будет доступен неявный конструктор по умолчанию.

Поведение неявного конструктора по умолчанию:
– поля примитивных типов **не инициализируются**;
– если в качестве поля выступает сложный объект, будет вызван его конструктор по умолчанию.

⚠ Если в классе присутствует поле примитивного типа, всегда объявляйте хотя бы один явный конструктор для гарантированной инициализации объекта

ООП на Си++. Спецификаторы доступа

```
struct Account {
private:
  int balance;

public:
  Account() {
    balance = 0;
  }

  int get_balance() {
    return balance;
  }

  void put(int amount) {
    balance += amount;
  }

  bool withdraw(int amount) {
    …
  }
};
```

```
int main() {
  Account account;
  account.put(100);

  int balance = account.balance;
  int balance = account.get_balance());

  bool r1 = account.withdraw(60);
  bool r2 = account.withdraw(60);
}
```

public – открытые члены класса (к полю/методу может обратиться любой внешний код)

private – закрытые члены класса (к полю/методу может обратиться только код из методов этого же класса)

⚠ Данные всегда должны быть закрытыми

ООП на Си++. Спецификаторы доступа

```
struct Account {
  // public:
  …
};
```

```
class Account {
  // private:
  …
};
```

ООП на Си++. Разделение на объявление и определение

```cpp
/* account.h */

struct Account {
private:
  int balance;

public:
  Account();

  bool withdraw(int amount);
};
```

```cpp
/* account.cc */

#include "account.h"

Account::Account() {
  balance = 0;
}

bool Account::withdraw(int amount) {
  …
}
```

ООП на Си++. Соглашение об именах

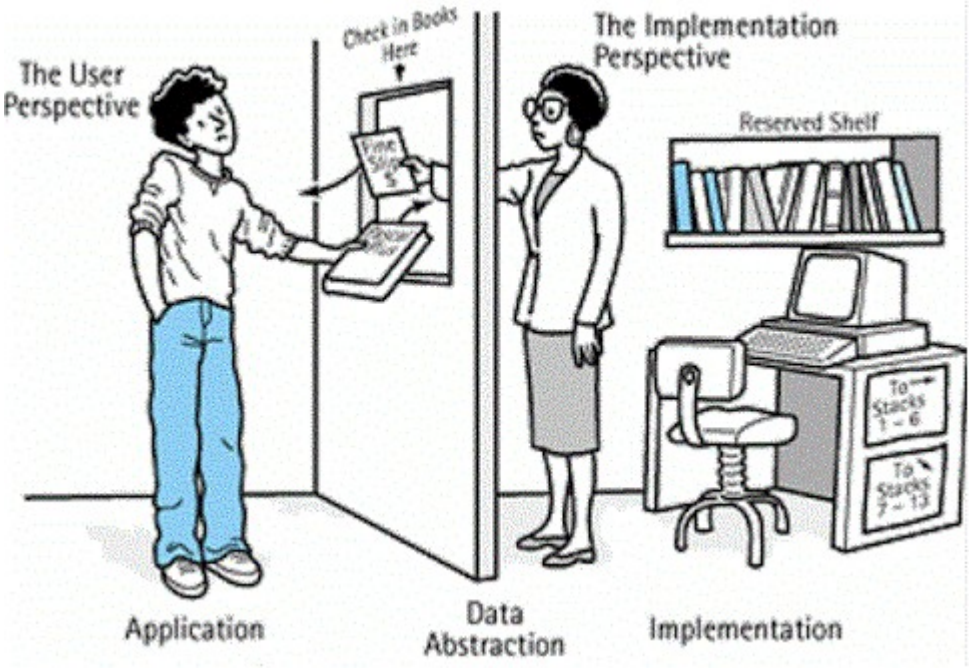
```
            Файлы: snake_case
Пространства имён: snake_case
          Классы: CamelCase
            Поля: snake_case или _snake_case
         Методы: snake_case
     Переменные: snake_case
```

1. Style Guide for Python Code
2. Google C++ Style Guide
3. C# Coding Conventions

Заключение

Три кита ООП

1. **Инкапсуляция**
(сокрытие деталей реализации)
API – Application Programming Interface

2. Наследование

3. Полиморфизм